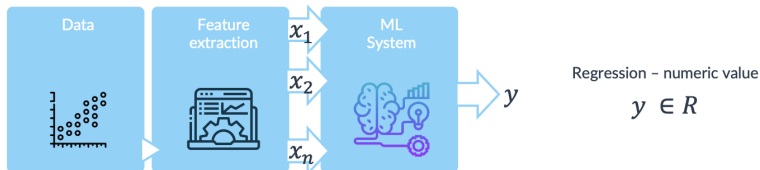


Supervised learning: generalization

Stefano Diciotti¹

¹Dipartimento di Ingegneria dell'Energia Elettrica e dell'Informazione "Guglielmo Marconi" - DEI, Università di Bologna

Supervised Learning: Linear regression

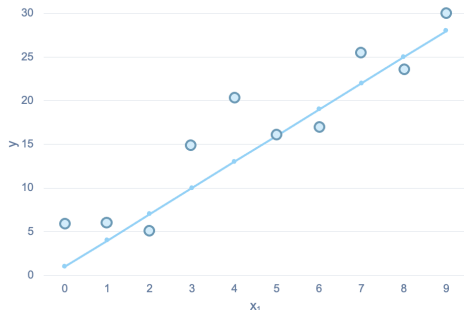


n : number of features

m : number of subjects (samples)



Linear Regression (1 Feature, No Bias)



Linear regression with one input feature
(no bias term, passes through the origin)

$$\hat{y} = w_1 x_1$$

where:

$$x_1 \in \mathbb{R}, \quad w_1 \in \mathbb{R}$$

Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

Linear Regression (2 and >2 Features, No Bias)

Linear regression with two input features and no bias term:

$$\hat{y} = w_1 x_1 + w_2 x_2$$

Vector notation:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \in \mathbb{R}^2, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \in \mathbb{R}^2$$

$$\hat{y} = \mathbf{w}^\top \mathbf{x}$$

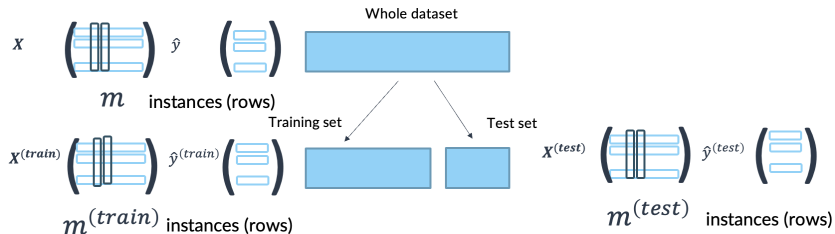
General case with n features:

$$\mathbf{x} \in \mathbb{R}^n, \quad \mathbf{w} \in \mathbb{R}^n, \quad \hat{y} = \mathbf{w}^\top \mathbf{x}$$

Both $\mathbf{x}$ and $\mathbf{w}$ are column vectors.

$\mathbf{w}$ is the vector of parameters (i.e., the set of weights)

Linear Regression: Data Split



where

$$m^{(train)} + m^{(test)} = m$$

Training objective: minimize the Mean Squared Error on the training set

$$\text{MSE}_{\text{train}} = \frac{1}{m^{(\text{train})}} \sum_{i=1}^{m^{(\text{train})}} \left(y_i^{(\text{train})} - \hat{y}_i^{(\text{train})} \right)^2 = \frac{1}{m^{(\text{train})}} \sum_{i=1}^{m^{(\text{train})}} \left(y_i^{(\text{train})} - \mathbf{w}^\top \mathbf{x}_i \right)^2$$

To minimize $\text{MSE}_{\text{train}}$, set the gradient to zero:

$$\nabla_{\mathbf{w}} \text{MSE}_{\text{train}} = 0 \quad \Rightarrow \quad \textbf{Normal equations:} \quad \mathbf{X}^{(\text{train})\top} \mathbf{X}^{(\text{train})} \mathbf{w} = \mathbf{X}^{(\text{train})\top} \mathbf{y}^{(\text{train})}$$

Explicit solution:

$$\mathbf{w} = \left(\mathbf{X}^{(\text{train})\top} \mathbf{X}^{(\text{train})} \right)^{-1} \mathbf{X}^{(\text{train})\top} \mathbf{y}^{(\text{train})}$$

(Optional demonstration: Chapter 5 of Deep Learning by Ian Goodfellow et al.)

Why is it a Minimum? (1 Feature, No Bias)

The MSE loss is convex in w

$$\text{MSE}_{\text{train}}(w) = \frac{1}{m} \sum_{i=1}^m (y_i - wx_i)^2$$

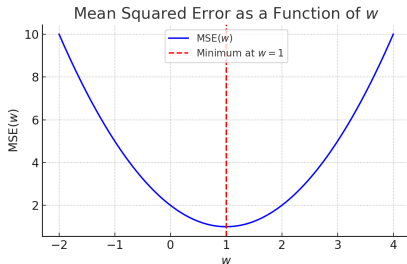
It has the shape of a parabola.

Minimum is found where the derivative is zero:

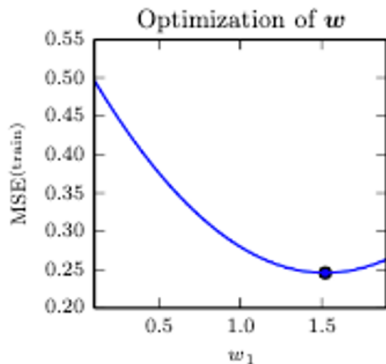
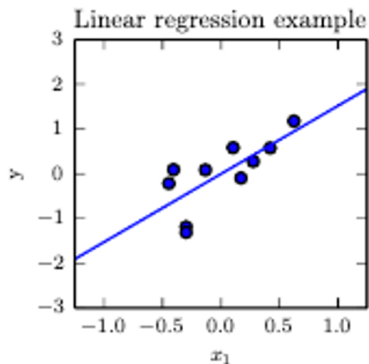
$$\frac{d}{dw} \text{MSE}_{\text{train}}(w) = 0$$

This is the global minimum since the function is convex.

Note: a function is convex if any line segment between two points on the graph lies above or on the graph itself.



Linear regression: Training (1 Feature, No Bias)



Linear Regression with Intercept Term (Bias)

Adding a bias term leads to an affine transformation

One feature:

$$\hat{y} = w_1 x_1 + b \quad \text{with} \quad x_1 \in \mathbb{R}, w_1, b \in \mathbb{R}$$

Equivalent formulation: adding a constant feature

Add a new input feature $x_2 = 1$, and write:

$$\hat{y} = w_1 x_1 + b = w_1 x_1 + w_2 \cdot 1 = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$$

where

$$\tilde{\mathbf{x}} = \begin{bmatrix} x_1 \\ 1 \end{bmatrix} \in \mathbb{R}^2, \quad \tilde{\mathbf{w}} = \begin{bmatrix} w_1 \\ b \end{bmatrix} \in \mathbb{R}^2$$

(bias as the last element of the vector)

General case with n features:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}, \quad \text{with} \quad \tilde{\mathbf{w}} \in \mathbb{R}^{n+1}$$

where

$$\tilde{\mathbf{x}} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \\ 1 \end{bmatrix} \in \mathbb{R}^{n+1}, \quad \tilde{\mathbf{w}} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \\ b \end{bmatrix} \in \mathbb{R}^{n+1}$$

Generalization and Test Error

Central challenge:

The machine learning algorithm must perform well on new, previously unseen inputs. This ability is called **generalization**.

Generalization error, also called the **test error**, must be low as well.

Mean Squared Error on test set:

$$\text{MSE}_{\text{test}} = \frac{1}{m^{(\text{test})}} \sum_{i=1}^{m^{(\text{test})}} \left(y_i^{(\text{test})} - \hat{y}_i^{(\text{test})} \right)^2$$

The data come from an underlying probability distribution

In supervised learning, we assume that both the training and test data are generated from a probability distribution over datasets (i.e., input-output pairs (x, y)), called the **data-generation process**.

In simple terms:

Every time we build a new dataset, we get a different set of instances — but all are sampled from the same underlying distribution $P(x, y)$.

Clinical example:

Each time a new patient arrives in a hospital, their data (e.g., age, blood pressure, diagnosis, outcome) are generated by a complex combination of biological, environmental, and behavioral factors.

This process is unknown and stochastic, but we assume that:

- Each patient instance is sampled independently
- Training and test sets come from the same distribution $P(x, y)$

Generalization: Data-Generation Process and i.i.d. Assumptions

We typically assume that data satisfy the **i.i.d. assumptions** (independent and identically distributed):

- **Independent:** each instance in the dataset is generated independently from the others
- **Identically distributed:** training and test instances are drawn from the same probability distribution

In other words: both sets contain the same type of data, with similar statistical properties (e.g., mean, variance, structure).

Note: if two instances represent data from the same patient at different time points, this violates the independence assumption due to temporal correlation.

If the test set comes from a different population (e.g., different hospital, machine, time period): *the data are not identically distributed* → this is known as a **distribution shift**, a common real-world challenge.

Result:

Under the i.i.d. assumptions, the expected training error of a randomly selected model is equal to its expected test error.

Generalization: Training vs. Test Error - Underfitting and Overfitting

When using a machine learning algorithm, we do not fix the model parameters in advance and then sample data. Instead, we:

- 1 Sample a **training set**
- 2 Use it to choose the model parameters by minimizing training error
- 3 Then evaluate on a separately sampled **test set**

Important: Under this process, the expected test error is **greater than or equal to** the expected training error.

A good algorithm should:

- Make the training error small
- Keep the gap between training and test errors small

Two central challenges in machine learning:

- **Underfitting:** training error remains high (poor fit to data)
- **Overfitting:** training error is low, but test error is much higher

Generalization: Model Capacity and Hypothesis Space

We can control whether a model is more likely to **overfit** or **underfit** by adjusting its **capacity**.

Model capacity: the ability of a model to fit a wide variety of functions

- Low-capacity models may struggle to fit the training data → **underfitting**
- High-capacity models may memorize irrelevant patterns in the training data → **overfitting**

We control model capacity by choosing the **hypothesis space** of the learning algorithm.

Hypothesis space: the set of functions the algorithm is allowed to choose from.

Example:

- In linear regression, the hypothesis space contains all linear functions of the input
- We can increase capacity by extending the hypothesis space to include polynomial functions

Generalization: Polynomial Regression Models

Polynomial of degree 1 (linear regression model):

$$\hat{y} = b + wx$$

Let us introduce an x^2 term (quadratic as a function of x):

$$\hat{y} = b + w_1x + w_2x^2$$

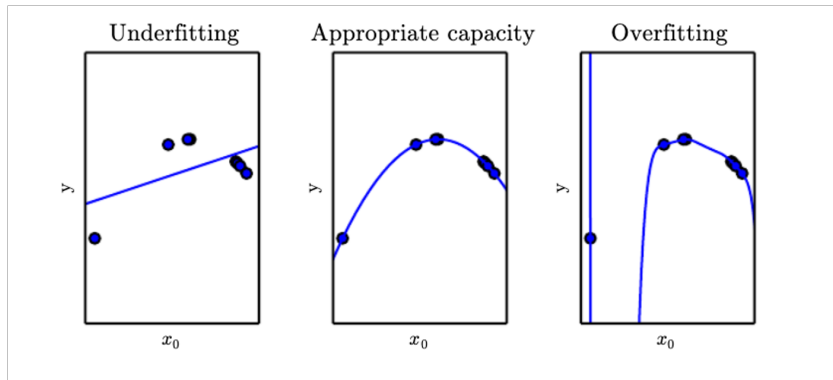
Though this model is a quadratic function of the input, the output is still a **linear function of the parameters**, so we can still use the normal equations to train the model in closed form.

We can add more powers of x as additional features, for example:

To obtain a polynomial of degree 9:

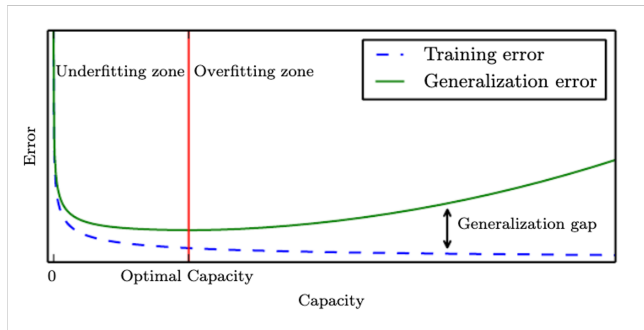
$$\hat{y} = b + \sum_{i=1}^9 w_i x^i$$

Generalization: Example



We saw that changing the number of features and, at the same time, adding new parameters associated with those features is a way to change a model's **capacity**.

Generalization: Underfitting and Overfitting



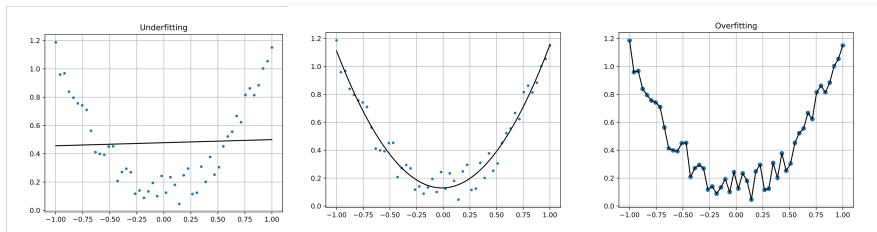
Underfitting

Poor performance on the training data and poor generalization to other data

Overfitting

Good performance on the training data, poor generalization to other data

Generalization: Underfitting and Overfitting - Regression

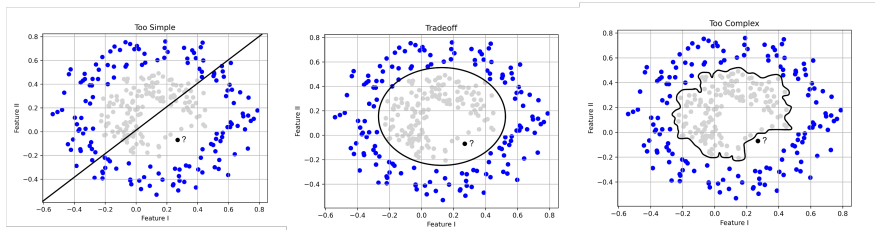


Underfitting

Model capacity (complexity)

Overfitting

Generalization: Underfitting and Overfitting - Classification



Underfitting

Model capacity (complexity)

Overfitting

Occam's Razor (William of Ockham)

"Among competing models that explain the data equally well, the one with the fewest assumptions should be selected."

In machine learning, this is known as the **principle of parsimony**: prefer simpler models unless more complex ones provide significantly better generalization.

Why?

- Simpler models are less likely to overfit
- They tend to generalize better on unseen data
- They are easier to interpret and debug

Reminder: increasing model capacity without necessity can harm generalization.



William of Ockham (c. 1287–1347)

- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, Chapter 5

<https://www.deeplearningbook.org/>